

```

/**
 * server/lib/token-hourly-cap.ts
 *
 * Fix #1 – Token Budgets Exhaust in 6 Minutes After Every Reboot
 *
 * PROBLEM: Daily token budgets (content-grinder: 500k, shorts-pipeline: 150k)
 * are stored in DB and persist across reboots. On each boot, 20+ services
 * hammer AI simultaneously, burning through the remaining daily budget in
 * minutes. After that, every pipeline AI call fails for the rest of the day.
 *
 * SOLUTION: Add a per-hour cap on top of the daily cap. Each module can use
 * at most MAX_TOKENS_PER_HOUR tokens per hour, regardless of daily remaining.
 * This spreads consumption across the full day so no single boot burst can
 * exhaust the budget.
 *
 * How it works:
 *   - Tracks tokens used per module in the current hour window
 *   - Resets the hourly counter when the hour rolls over
 *   - Returns { allowed: false } if the hourly cap would be exceeded
 *   - Falls through to the existing daily budget check if allowed
 *
 * Integration: call checkHourlyTokenBudget() at the TOP of each module's
 * AI call site, BEFORE calling callOpenAI/callClaude.
 */

```

```
import { createLogger } from './logger.js';
```

```
const log = createLogger('token-hourly-cap');
```

```

// — Per-module hourly caps —————
// Set to ~20% of daily budget so a full day can distribute evenly
// Adjust these based on observed burn rates

```

```

export const HOURLY_CAPS: Record<string, number> = {
  'content-grinder':      50_000,    // daily 500k ÷ 24h × 2.4 buffer
  'shorts-pipeline':      12_000,    // daily 150k ÷ 24h × 1.9 buffer
  'repurpose-engine':       8_000,
  'thumbnail-intelligence': 6_000,    // daily 100k ÷ 24h × 1.4 buffer
  'viral-optimizer':      8_000,    // daily 150k ÷ 24h × 1.3 buffer
  'vod-seo-optimizer':    6_000,
  'infinite-evolution':   4_000,
  'knowledge-mesh':       3_000,
  'self-improvement-engine': 3_000,
  'autonomous-capability': 4_000,
  'memory-architect':     3_000,
  'business-agents':      2_000,
  'legal-tax-agents':     2_000,
  'team-orchestration':   3_000,

```

```

'growth-flywheel':      3_000,
'consistency-agent':    3_000,
'default':              5_000,    // fallback for unlisted modules
};

// ——— In-memory hourly tracking ———
// Keyed by `${moduleName}:${hourKey}` where hourKey = UTC hour integer (0-23)

interface HourlySlot {
  hourKey: number;
  usedTokens: number;
}

const hourlyUsage = new Map<string, HourlySlot>();

function getCurrentHourKey(): number {
  return Math.floor(Date.now() / (60 * 60 * 1000)); // hours since epoch, always increases
}

function getSlot(module: string): HourlySlot {
  const currentHour = getCurrentHourKey();
  const existing = hourlyUsage.get(module);

  if (existing && existing.hourKey === currentHour) {
    return existing;
  }

  // Hour rolled over or first access – reset slot
  const fresh: HourlySlot = { hourKey: currentHour, usedTokens: 0 };
  hourlyUsage.set(module, fresh);
  return fresh;
}

// ——— Public API ———

export interface TokenCapResult {
  allowed: boolean;
  usedThisHour: number;
  hourlyLimit: number;
  remaining: number;
}

/**
 * Check if a module can make an AI call without exceeding its hourly cap.
 * Call this BEFORE every AI call in a module.
 *
 * Usage:
 *   const check = checkHourlyTokenBudget('content-grinder', estimatedTokens);
 *   if (!check.allowed) {
 *     log.warn(`[ContentGrinder] Hourly token cap reached – deferring`);

```

```

*    return;
*    }
*    // proceed with AI call
*/

export function checkHourlyTokenBudget(
  module:      string,
  estimatedTokens: number = 1000,
): TokenCapResult {
  const cap    = HOURLY_CAPS[module] ?? HOURLY_CAPS['default'];
  const slot    = getSlot(module);
  const after   = slot.usedTokens + estimatedTokens;

  if (after > cap) {
    log.debug(
      `[HourlyCap] ${module} would exceed hourly cap: ` +
      `${slot.usedTokens}+${estimatedTokens}=${after} > ${cap}`
    );
    return {
      allowed:      false,
      usedThisHour: slot.usedTokens,
      hourlyLimit:  cap,
      remaining:    Math.max(0, cap - slot.usedTokens),
    };
  }

  return {
    allowed:      true,
    usedThisHour: slot.usedTokens,
    hourlyLimit:  cap,
    remaining:    cap - slot.usedTokens,
  };
}

/**
 * Record tokens actually used after a successful AI call.
 * Call this AFTER every AI call completes.
 *
 * Usage:
 *   const result = await callOpenAI({ ... });
 *   recordHourlyTokenUsage('content-grinder', result.usage?.total_tokens ?? 1000);
 */

export function recordHourlyTokenUsage(module: string, tokensUsed: number): void {
  const slot = getSlot(module);
  slot.usedTokens += tokensUsed;
}

/**
 * Snapshot of all module hourly usage for diagnostics.
 * Expose via /api/health.
 */

```

```
export function getHourlyCapStatus(): Record<string, {
  used: number;
  limit: number;
  pct: number;
}> {
  const result: Record<string, { used: number; limit: number; pct: number }> = {};
  const currentHour = getCurrentHourKey();

  for (const [module, cap] of Object.entries(HOURLY_CAPS)) {
    const slot = hourlyUsage.get(module);
    const used = (slot && slot.hourKey === currentHour) ? slot.usedTokens : 0;
    result[module] = { used, limit: cap, pct: Math.round((used / cap) * 100) };
  }

  return result;
}
```